Introduction to GitHub Copilot

Define, inspect, test, decide

Session M1-01 | 3 hours

Agenda

Segment	Time
Suggestion-and-tests model	7 min
How context shapes a suggestion	10 min
Inline suggestion controls	15 min
Prepared utility demonstration	18 min
Follow-up edits	6 min
Access check and lab handoff	4 min

The review workflow

```
Define observable behavior
          ↓
Inspect the suggestion
          ↓
Run focused checks
          ↓
Accept, revise, or reject
```

The developer owns the result.

Plausible is not correct

Looks right	Is right
Reads well	A focused test passes
Uses familiar syntax	The diff stays in scope
Matches one example	Edge and failure cases are checked

A practical system model

```
Task + context
          ↓
Copilot surface
          ↓
Suggestion
          ↓
Tests + human review
```

Context changes the suggestion

Useful context includes:

- function names and type hints;
- docstrings and examples;
- selected code and nearby files;
- tests and repository instructions.

Accept, revise, reject

Accept when the proposal meets the contract and fits the repository.

Revise when the direction is useful but a detail is wrong.

Reject when it changes scope, invents behavior, or is hard to check.

Undo is a normal review action.

Review one suggestion

Before accepting, ask:

- Which requirement does each line support?
- What happens on empty or unusual input?
- Did it add a dependency?
- Did it change a caller or public name?
- Which check will fail if the logic is wrong?

Demo contract

```
def normalize_username(value: str) -> str:
    """Trim outer whitespace, lowercase the value,
    and join words with hyphens."""
```

```
assert normalize_username("  River Team  ") == "river-team"
```

The test checks how repeated whitespace is handled.

Demo: inspect before accepting

Review the proposed implementation against three rules:

1. Outer whitespace disappears.
2. Case becomes lowercase.
3. Each run of internal whitespace becomes one hyphen.

Then run the focused test.

A failing test is useful

This version looks reasonable but mishandles repeated spaces:

```
return value.strip().lower().replace(" ", "-")
```

The test catches it while the change is still small.

Predicted follow-up edits still need review

```
Rename username to handle
          ↓
Inspect each proposed caller update
          ↓
Search for the old name
          ↓
Run the focused test
```

A predicted edit can miss a reference.

Access check

Every learner must be able to request an inline suggestion before the lab starts.

Lab handoff

You will extend one Python utility module:

1. complete `normalize_username` and `is_palindrome`;
2. decide how apostrophes count in `word_frequency`;
3. add `profile_summary`;
4. rename `username` to `handle`;
5. run the full test suite and review the diff.

Deliverable: completed code, five passing tests, and a decision record.